Übung 1 - Praktikum ASC Lösung

Aufgabe 1

Schreiben Sie ein TiEL-Programm, das mittels der Leibniz-Reihe eine Näherung von π berechnet.

Erinnerung: $\sum_{k=0}^{\infty} \frac{(-1)^k}{2k+1} = \frac{\pi}{4}$

Lösung:

```
1  var pi = 0;
2  var sign = 1;
3  var n = 0;
4  var term=0;
5
6  while n < 10000 {
7      term = sign / (2 * n + 1);
8      pi = pi + term;
9      sign = -sign;
10     n = n + 1;
11 }
12 print(4*pi);
```

Aufgabe 2

Schreiben Sie eine Funktion, die herausfindet, ob zwei natürliche Zahlen a und b befreundet sind. Dies ist der Fall, wenn die Summe der echten Teiler von a b ergibt und umgekehrt.

Hinweis: Es gibt keinen Modulo-Operator, stattdessen können Sie die Funktion `floor` benutzen, die den Ganzzahlanteil einer Zahl zurückgibt.

Lösung:

```
1  fun teilersumme(n, teiler) {
2    if teiler==n { return 0; }
3    if (n-floor(n/teiler)*teiler==0)  // teilbar
4       { return teiler+ teilersumme(n, teiler+1); }
5    else { return teilersumme(n, teiler+1); }
6  }
7
8  fun befreundet(a,b) {
9  var ta=teilersumme(a,1);
10 var tb = teilersumme(b,1);
11 print("teilersumme("); print(a);print(")="); print(ta);
12 print("teilersumme("); print(b);print(")=");print(tb);
13   if (teilersumme(a,1)==b) and  (teilersumme(b,1)==a)
14    { return true; }
15   else { return false; }
16 }
17 var n=220;
18 var m=284;
19 print(befreundet(n, m));
```

Aufgabe 3

Schreiben Sie ein Programm, das die beiden kleinsten Zahlen findet, die befreundet sind.

Lösung: Die kleinsten befreundeten Zahlen sind 220 und 284.

Aufgabe 4

Schreiben Sie eine Funktion, die die Hexanacci-Zahlen berechnet

$$h(n) = \begin{cases} 0 & , falls\ n = 0 \\ 2^{n-1} & , falls\ 1 \leq\ n \leq\ 5 \\ h(n-1) + \ldots h(n-6) & , sonst \end{cases}$$

Kleine Knobelaufgabe: Was hat das mit Würfeln zu tun?

Lösung: Hexanacci-Zahlen zählen die Anzahl der geordneten Summen, die man mit einer beliebigen Anzahl von Würfen eines sechsseitigen Würfels bilden kann, sodass die Summe n ergibt.

Beispiel: Die Zahl 4 kann man würfeln mit 4, 1+3, 3+1, 2+2, 1+1+2, 1+2+1, 2+1+1, 1+1+1+1, das sind 8 Möglichkeiten.

```
1  fun hex(n) {
2      if n == 0  { return 0; }
3      elif n == 1  { return 1; }
4      elif n==2  {return 2; }
5      elif n==3  {return 4;}
6      elif n==4  {return 8;}
7      elif n==5  {return 16;}
8      else {return hex(n - 1) + hex(n - 2) + hex(n-3) + hex(n-4) +
           hex(n-5);}
9  }
10 var i = 0;
11 while i < 20  {
12     print(hex(i));
13     i = i + 1;
14 }
```

Aufgabe 5

Schreiben Sie ein Programm, das für eine gegebene Zahl die Collatz-Folge ausgibt. Diese Folge ist definiert durch:

$$c_{n+1} = \begin{cases} 1 & , falls\ c_n = 1 \\ c_n/2 & , falls\ c_n\ gerade \\ 3 \cdot c_n + 1 & , sonst \end{cases}$$

Beispiel: Für den Startwert 9 ergibt sich die Folge 9, 28, 14, 7, 22, 11, 34, 17, ..., 1

Lösung:

```
1  fun c(n) {
2    print(n);
3    if n==1 { return 1; }
4    if (n-floor(n/2)*2==0)   // teilbar
```

```
5       { return c(floor(n/2)); }
6     else { return c(3*n+1); }
7   }
8
9   var n=9;
10  c(n);
```